

Spike: Unifying the Landing Page and Intake App Under One Domain

Status: spike / research only — no code changes. Prepared: 2026-07-30

1. Current state

The product is one thing to the consultant but three separately-hosted pieces to a visitor:

1. The marketing landing page — Next.js on Vercel, currently on a *.vercel.app URL (no custom domain purchased yet). Has the "Start My Plan" CTA.
2. A tiny static "loading" page — Render static site at

career-transition-loading.onrender.com, whose only job is to poll career-transition-intake.onrender.com/_health (working around Render's free-tier cold-start) and then redirect the browser once it responds. The CTA currently links straight to this.

3. The actual intake Flask app — career-transition-intake.onrender.com, serving the form and handling POST /submit.

So today a click on "Start My Plan" bounces the visitor across three hostnames, two of them raw onrender.com subdomains. The goal is for this to feel like one product/one domain — no visible host switch — and to figure out whether a custom domain needs buying and where it should point.

2. Options considered

2.1 DNS subdomain split (same registered domain, no proxy)

Buy the domain, point the apex/www at Vercel (standard "Add Domain" flow), and point a subdomain (e.g. app.) at Render via CNAME — Render's free tier supports custom domains on web services. No app or infra code changes beyond swapping the CTA's href.

This is the lowest-effort option by far (~15 minutes, pure DNS) but only partially solves the problem: the address bar still visibly changes on click (careertransition.co.ke → app.careertransition.co.ke), so a user who reads the URL bar notices a hop. What it does fully solve is the actual "looks unprofessional" complaint — no more onrender.com leaking into what the visitor sees, since everything now lives under one recognizable brand domain. It does not touch the cold-start problem; the loading page (or something like it) is still needed.

2.2 Vercel rewrite proxying to Render

A vercel.json rewrite (destination: "https://career-transition-intake.onrender.com/:path*") can proxy a path like /apply/* on the primary domain straight through to the Render Flask app at Vercel's edge, keeping the browser's address bar on one hostname throughout. This also gives a home for retiring the standalone loading-page host — it can become a path under the main domain too, or be dropped if cold start is fixed directly (see §2.4).

The one real unknown: Vercel's documented 4.5MB request-body limit is explicit for Vercel Functions, and the intake form needs to support 10MB uploads. External-origin rewrites use a separate proxy mechanism with their own timeout and error code (ROUTER_EXTERNAL_TARGET_ERROR, hard 120s cap on

first-byte), which suggests — but doesn't explicitly confirm in Vercel's own docs — that the 4.5MB Function limit may not apply to pure external rewrites. This needs an empirical test (push a >4.5MB multipart POST through a rewrite in a preview deployment) before relying on it. The 120s timeout is also worth watching: it's survivable against Render's ~60s cold-start wake-up, but tight if a cold start and a slow `/submit` (PDF build + Resend call) stack on the same request.

Multipart bodies pass through fine in principle (rewrites proxy framework-agnostic bytes), and there's no extra cost — rewrites are a config-only feature on Vercel's Hobby plan.

Effort: low — one config change to the existing Next.js project, plus the domain purchase. No changes needed to the Flask app itself.

2.3 Reverse proxy in front of both (e.g., a Cloudflare Worker)

Move the domain's DNS to Cloudflare (free plan) and run a Worker that routes `/apply/*` to Render and everything else to Vercel. This sidesteps the body-size ambiguity in §2.2 entirely — a Worker just forwards raw bytes with no separate Function-layer limit to worry about — and Cloudflare's proxy timeout/streaming behavior is more generous than Vercel's 120s cap.

The tradeoff is more moving parts: three parties instead of two (Cloudflare, Vercel, Render), DNS management moves off Vercel's own nameservers, and there's a Worker script to write and own going forward. Still comfortably within Cloudflare's free tier (100k requests/day) at this scale.

Effort: medium — Worker code + DNS cutover, more setup than §2.2 but no ambiguity to test around.

2.4 Migrate the Flask app onto Vercel, eliminating Render entirely

Vercel auto-detects Flask (via `requirements.txt` + an app object) and compiles it into a single Vercel Function on Fluid Compute. `/tmp` is writable up to 500MB per invocation (matches the current build-PDF-then-stream-or-email pattern), duration limits (300s default, up to 800s+ on Pro) comfortably cover PDF generation + the Resend call, and cold starts on Python Fluid Compute are sub-second to low-seconds — which would eliminate the entire reason the loading page exists, as a side effect.

Two real blockers, though:

- ~~**~~The 4.5MB request-body limit is explicit and confirmed for Vercel

Functions* — and a Flask app deployed to Vercel is* a Function. The stated 10MB upload requirement exceeds this outright. Working around it means restructuring uploads to go client-side directly to Vercel Blob storage, with the Function only receiving a blob reference — a real architecture change, not a config toggle.

- Flask-Limiter's default in-memory storage doesn't work on serverless.

MemoryStorage is documented as "for testing only" and isn't shared across instances — on Vercel's auto-scaling Fluid Compute, concurrent invocations landing on different instances would silently give each its own rate-limit counter. Fixing this means pointing `storage_uri` at Redis (e.g., via the Vercel Marketplace's Upstash integration) — a new piece of infrastructure.

Effort: medium-high — not a lift-and-shift; the upload-size limit forces a genuine re-architecture, plus a new Redis dependency for rate limiting. Worth doing eventually (it's the only option that also kills the cold-start problem outright), but not a starting point.

2.5 Vercel Microfrontends / multi-zone — ruled out

Vercel's Microfrontends feature is explicitly Vercel-project-to-Vercel- project only; its own docs, for a backend not yet on Vercel, say to create an intermediary Vercel project that rewrites to the external host — which is just \$2.2 with an extra layer of dashboard config and no functional benefit. Not worth pursuing here.

3. Comparison

Option	Effort	Ongoing maintenance	Solves visible host-switch?	Also fixes cold start?
2.1 DNS subdomain split	Lowest (~15 min)	Minimal (domain renewal)	Partial — same brand domain, but the URL still changes on click	No
2.2 Vercel rewrite → Render	Low (one config file)	Low, but verify body-size behavior first and watch the 120s proxy timeout	Full, if the upload test passes	No
2.3 Cloudflare Worker in front of both	Medium (Worker + DNS cutover)	Medium — one more system to own	Full, no size ambiguity	No
2.4 Migrate Flask → Vercel	Medium-high (re-architect uploads + add Redis)	Higher near-term, lower long-term	Full, and removes Render as an entity entirely	Yes
2.5 Microfrontends	—	—	Reduces to 2.2 with no added benefit	—

4. Recommendation

1. Buy the domain now regardless — every option needs it, and it's the cheapest, lowest-risk step to take first.

2. Try 2.2 (Vercel rewrite to Render) first. One config change, no new vendor, and it directly kills the visible host-switch for both hops. Before committing, spend under an hour testing a >4.5MB multipart POST through a rewrite in a preview deployment — this resolves the one real unknown and tells you definitively whether it's viable as-is.

3. If that test fails, fall back to 2.3 (Cloudflare Worker). It sidesteps the body-size question entirely, at the cost of moving DNS and writing a small Worker script — still low-effort, high-certainty.

4. Treat 2.1 (DNS subdomain split) as the same-day fallback, not the target state — cheapest possible improvement (kills the onrender.com branding leak) but doesn't fully achieve one URL throughout. Fine as an interim step while 2.2/2.3 is being tested.

5. Defer 2.4 (migrate Flask → Vercel). It's the only option that also solves cold start (making the loading page unnecessary), which is attractive, but the 4.5MB body limit directly conflicts with the 10MB upload requirement and forces a real architecture change plus a new

Redis dependency. Worth revisiting as a phase 2 once domain unification is settled, or sooner if Render's cold-start/free-tier behavior becomes a recurring problem in its own right (it already interacts with the payment spike above — a webhook landing on a cold instance is a lost payment confirmation, not just a slow page load).

5. Open questions

- Domain name/TLD and registrar choice — not yet decided.
- Whether to address Render's cold start directly (upgrade off free tier, or

a keep-alive ping) regardless of which unification option is chosen, since it now also matters for payment-webhook reliability (see the payment integration spike), not just page-load UX.

- Whether the loading page's UX (spinner + poll + redirect) should be kept

under a path of the unified domain, or dropped once cold start is addressed directly.

Sources

- [Rewrites on Vercel](#)
- [Vercel Limits](#)
- [Vercel Functions Limits](#)
- [Vercel — Python Runtime](#)
- [Vercel — Deploy a Flask app](#)
- [Vercel — Can I use Vercel as a reverse proxy?](#)
- [Vercel — How to bypass the 4.5MB body size limit](#)
- [Render Docs — Free Tier](#)
- [Flask-Limiter documentation](#)